

IDENTIFICAÇÃO	
Disciplina: Paradigmas de Linguagem de Programação	Série: ESOFT6S-NA
ACADÊMICO	
RA: 23234248-2	Nome: Vítor Hugo da Costa Santos
LINK DO GITHUB	
https://github.com/vitorchibere/Paradigmas	

Relatório — Expressões, Atribuição e Controle

1. Introdução

Nesta aula foram apresentados alguns conceitos importantes sobre programação, principalmente relacionados a expressões, atribuição e controle do programa. Foram vistos assuntos como a ordem das operações, tipos de dados, conversão de tipos, expressões booleanas, estruturas de decisão e estruturas de repetição.

Durante a aula, também foram apresentados alguns exemplos mostrando que pequenos detalhes no código podem mudar o resultado. Nos exercícios isso ficou ainda mais claro, pois alguns códigos pareciam estar certos, mas tinham algum detalhe que fazia o programa funcionar de uma forma diferente do esperado.

Por isso, neste relatório, vou apresentar o que entendi dos principais conteúdos e relacionar com os exercícios que foram feitos, mostrando o que estava acontecendo e como cada correção se relaciona com o que foi aprendido.

2. Expressões e ordem das operações

Um dos primeiros assuntos da aula foi sobre expressões e a ordem em que as operações são realizadas.

Aprendi que os operadores não são executados simplesmente da esquerda para a direita. Existe uma ordem de prioridade entre eles, chamada de precedência. Também existe a associatividade, que ajuda a definir a ordem quando existem operadores com a mesma prioridade.

Um exemplo apresentado foi:

```
print(-2 ** 2, 2 ** 3 ** 2)
```

O resultado é:

```
-4 512
```

Isso acontece por causa das regras de precedência e associatividade da linguagem.

O que entendi com esse exemplo é que é importante conhecer essas regras, porque uma expressão pode ter um resultado diferente do que eu imaginaria olhando apenas rapidamente para o código.

3. Ordem de avaliação e efeitos colaterais

Também foi explicado que a ordem em que as partes de uma expressão são avaliadas pode influenciar o resultado.

Além disso, algumas operações podem causar efeitos colaterais, ou seja, além de gerar um resultado, também podem alterar alguma variável ou o estado do programa.

Essa parte me fez perceber que, principalmente em expressões maiores, é importante entender o que está sendo executado primeiro para não acabar interpretando o código de forma errada.

4. Sobrecarga de operadores

Outro assunto apresentado foi a sobrecarga dos operadores.

Um mesmo operador pode ter comportamentos diferentes dependendo dos tipos que estão sendo utilizados. O +, por exemplo, pode fazer uma soma quando estamos trabalhando com números, mas também pode juntar textos.

Um exemplo apresentado na aula foi:

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

O resultado é:

3345

Primeiro $1 + 2$ resulta em 3. Depois, quando aparece o texto "3", a operação passa a trabalhar com concatenação.

Com isso, entendi que não basta olhar somente para o operador. Também é preciso observar os tipos dos valores envolvidos.

5. Conversão de tipos

A aula também explicou a conversão de tipos.

Em algumas situações, a própria linguagem faz uma conversão automaticamente. Em outras, é necessário fazer a conversão de forma explícita.

Isso é importante porque trabalhar com tipos diferentes pode mudar o resultado de uma operação.

Um exemplo disso apareceu em um dos exercícios de Java, em que uma divisão entre números inteiros produzia um resultado diferente do esperado. Esse exercício ajudou a entender melhor esse assunto na prática.

6. Expressões booleanas

As expressões booleanas são usadas principalmente para verificar condições que podem ser verdadeiras ou falsas.

Também foi mostrado que algumas linguagens consideram determinados valores como verdadeiros ou falsos dentro de uma condição.

Um exemplo apresentado foi:

```
x = 0  
print(x or "padrão", 3 > 2 > 1)
```

O resultado é:

```
padrão True
```

Nesse caso, o 0 é considerado falso, então "padrão" acaba sendo utilizado.

Também foi apresentado o conceito de avaliação de curto-circuito, em que nem sempre todas as partes de uma expressão precisam ser avaliadas.

7. Atribuição

A atribuição é utilizada para colocar valores dentro das variáveis.

Um exemplo simples é:

```
x = 10
```

Também existem formas abreviadas, como:

```
x += 1
```

que possui a mesma ideia de:

```
x = x + 1
```

Apesar de parecer algo simples, os exercícios mostraram que é importante prestar atenção na forma como uma variável está sendo utilizada, principalmente quando existem diferentes tipos ou quando ela está dentro de algum bloco de código.

8. Estruturas de controle

Outro assunto importante da aula foram as estruturas de controle.

Elas servem para controlar o que o programa vai fazer dependendo de determinadas situações.

Foram apresentados exemplos de estruturas como `if`, `switch`, `for`, `while`, `break` e `continue`.

Nos exercícios, alguns desses comandos foram justamente os responsáveis pelos erros. Isso ajudou a entender melhor como eles funcionam, porque foi possível comparar o resultado antes e depois da correção.

9. Exercícios e relação com o conteúdo da aula

Exercício 1 — JavaScript

O primeiro exercício utilizava:

```
</> OneCompiler Upgrade AI JavaScript Run Save Logir
JS index.js x +
1 const precos = [10, 20, 30];
2
3 let total = 0;
4
5 for (const p in precos)
6   total += p;
7
8 console.log("Total: " + total);
```

Console I/O AI Agent 137 ms ✓

Total: 0012

No começo, esse resultado pode parecer estranho, já que os valores do vetor são 10, 20 e 30.

O problema está no `for...in`. Ele não pega diretamente os valores do vetor. Nesse caso, ele percorre as posições, que são 0, 1 e 2.

Além disso, esses valores acabam sendo tratados como texto na operação com `+`, fazendo acontecer uma concatenação em vez da soma esperada.

```
</> OneCompiler Upgrade AI JavaScript Run Save Logir
JS index.js x +
1 const precos = [10, 20, 30];
2
3 let total = 0;
4
5 for (const p of precos)
6   total += p;
7
8 console.log("Total: " + total);
```

Console I/O AI Agent 143 ms ✓

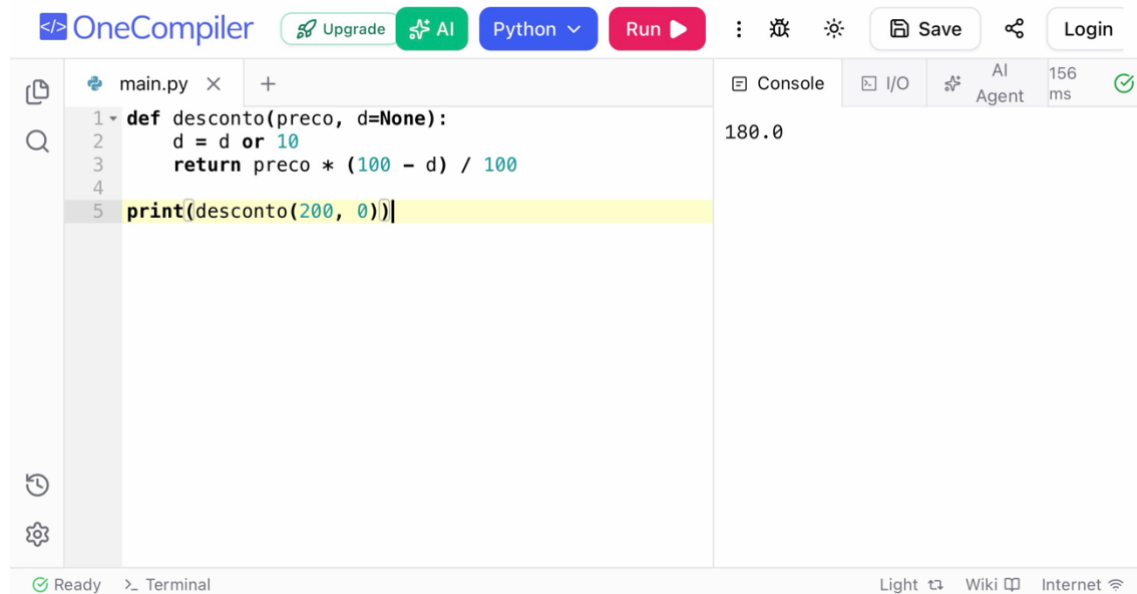
Total: 60

Ready > Terminal Light Wiki Internet

A principal coisa que entendi foi que `for...in` e `for...of` parecem parecidos, mas não fazem exatamente a mesma coisa. Também ficou mais fácil entender como o tipo do valor pode interferir no funcionamento do operador `+`.

Exercício 2 — Python

O segundo exercício foi:

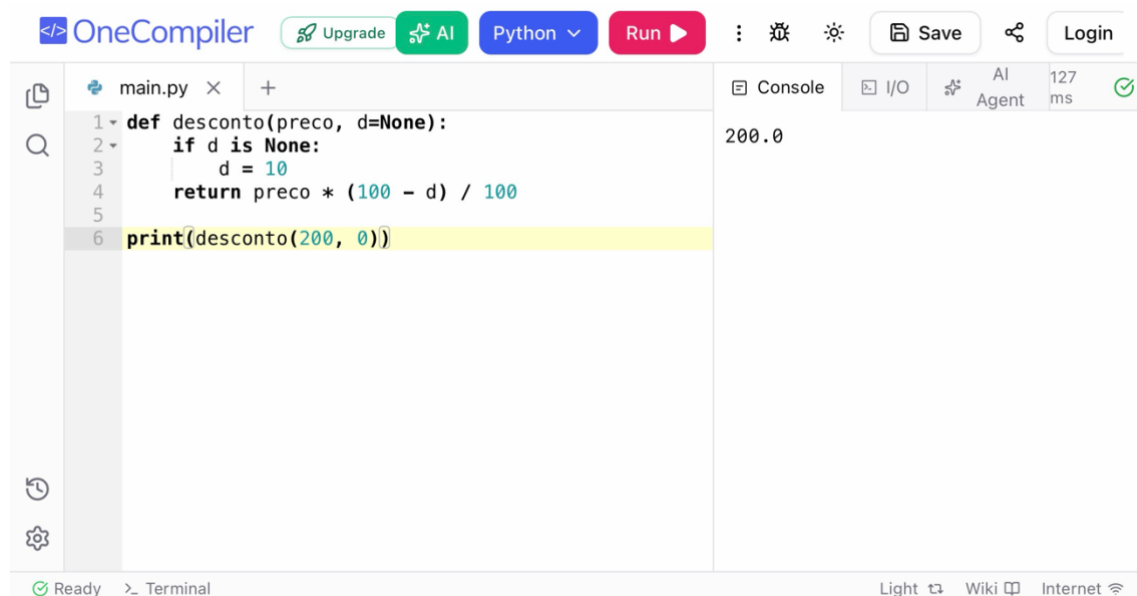


```
</> OneCompiler Upgrade AI Python Run Save Login
main.py x +
1 def desconto(preco, d=None):
2     d = d or 10
3     return preco * (100 - d) / 100
4
5 print(desconto(200, 0))
180.0
Ready >_ Terminal Light Wiki Internet
```

O problema está nesta parte:

`d = d or 10`

Como o valor 0 é considerado falso em uma expressão booleana, o programa entende que não existe um valor válido em `d` e coloca 10 no lugar.

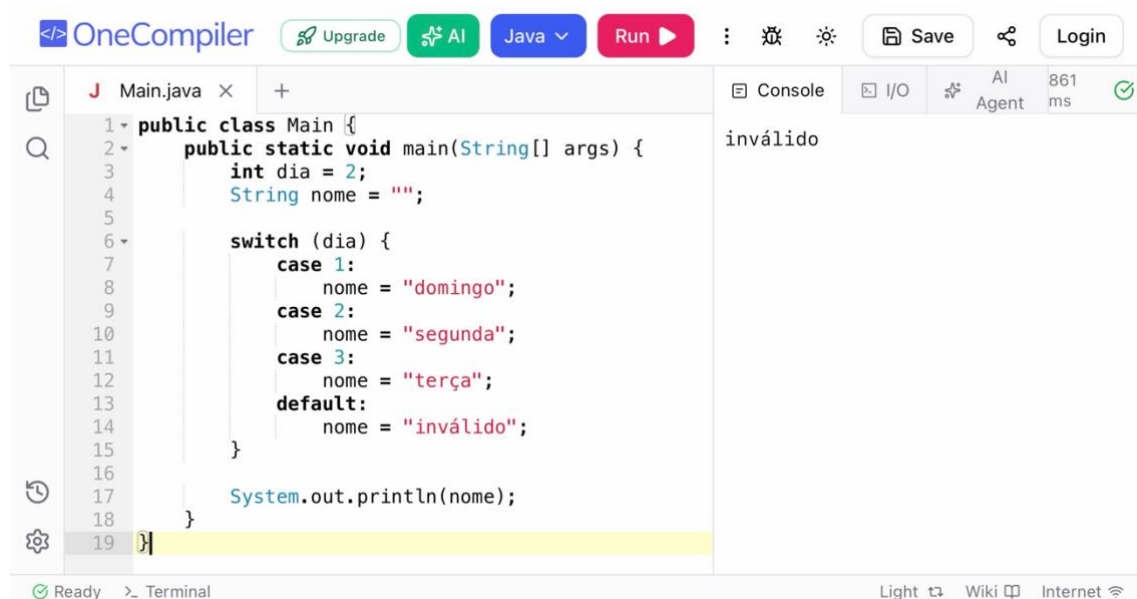


```
</> OneCompiler Upgrade AI Python Run Save Login
main.py x +
1 def desconto(preco, d=None):
2     if d is None:
3         d = 10
4     return preco * (100 - d) / 100
5
6 print(desconto(200, 0))
200.0
Ready >_ Terminal Light Wiki Internet
```

Esse exercício está diretamente relacionado ao conteúdo de expressões booleanas e avaliação de curto-circuito.

O que consegui entender melhor foi que 0 pode ser um valor válido para uma variável, mesmo sendo considerado falso em uma expressão lógica. Por isso, nesse caso, era melhor verificar especificamente se d era None.

Exercício 3 — Java



```
1 public class Main {
2     public static void main(String[] args) {
3         int dia = 2;
4         String nome = "";
5
6         switch (dia) {
7             case 1:
8                 nome = "domingo";
9             case 2:
10                nome = "segunda";
11            case 3:
12                nome = "terça";
13            default:
14                nome = "inválido";
15        }
16
17        System.out.println(nome);
18    }
19 }
```

Console: inválido

O problema estava na falta do break.

Quando o programa encontra o case 2, ele executa esse trecho, mas continua executando os próximos casos. No final, acaba chegando ao default e alterando o valor de nome.

The screenshot shows the OneCompiler IDE interface. At the top, there are buttons for 'Upgrade', 'AI', 'Java', 'Run', 'Save', and 'Login'. The main editor displays a Java file named 'Main.java' with the following code:

```
1 public class Main {
2     public static void main(String[] args) {
3         int dia = 2;
4         String nome = "";
5
6         switch (dia) {
7             case 1:
8                 nome = "domingo";
9                 break;
10            case 2:
11                nome = "segunda";
12                break;
13            case 3:
14                nome = "terça";
15                break;
16            default:
17                nome = "inválido";
18        }
19
20        System.out.println(nome);
21    }
22 }
```

On the right side, the 'Console' tab is active, showing the output 'segunda'. The status bar at the bottom indicates 'Ready', 'Terminal', 'Light' theme, 'Wiki', and 'Internet' connectivity. The execution time is shown as 909 ms.

Esse exercício se relaciona diretamente com o conteúdo de estruturas de seleção.

Foi uma forma prática de entender o funcionamento do switch e também perceber que o break é importante para impedir que o programa continue executando os próximos casos.

Exercício 4 — C

O quarto exercício tinha o seguinte código:

The screenshot shows the OneCompiler IDE with a C program in a file named Main.c. The code is as follows:

```
1 #include <stdio.h>
2
3 int main() {
4     int saldo = 100;
5     int saque = 50;
6
7     if (saque > saldo);
8
9     printf("saldo insuficiente\n");
10    printf("fim\n");
11
12    return 0;
13 }
```

The output in the console is:

```
saldo insuficiente
fim
```

The status bar at the bottom indicates 'Ready' and 'Terminal'.

O problema estava no ; colocado logo depois do if:

if (saque > saldo);

Esse ponto e vírgula faz com que o if tenha uma instrução vazia. Dessa forma, o printf que vem depois não fica realmente dentro da condição.

The screenshot shows the OneCompiler IDE with the same C program, but with the corrected code:

```
1 #include <stdio.h>
2
3 int main() {
4     int saldo = 100;
5     int saque = 50;
6
7     if (saque > saldo) {
8         printf("saldo insuficiente\n");
9     }
10
11    printf("fim\n");
12
13    return 0;
14 }
```

The output in the console is now:

```
fim
```

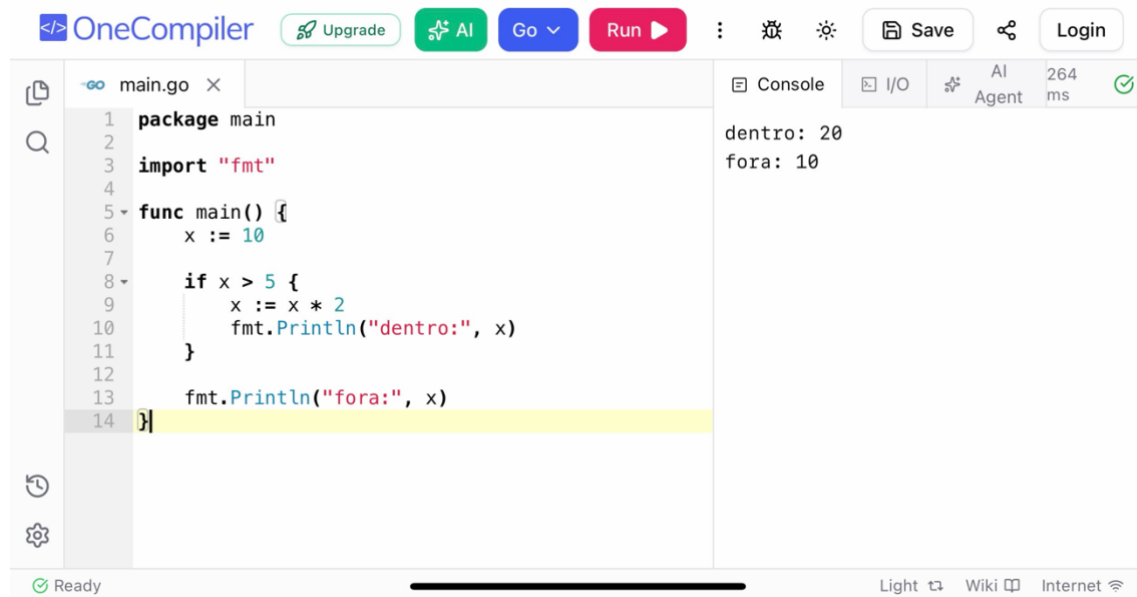
The status bar at the bottom indicates 'Ready' and 'Terminal'.

Esse exercício tem relação com as estruturas de controle, principalmente com o if.

O que mais chamou atenção foi que um detalhe muito pequeno, apenas um `;`, conseguiu mudar completamente o resultado do programa. Isso mostrou na prática a importância de prestar atenção na estrutura e na sintaxe do código.

Exercício 5 — Go

O quinto exercício foi:



The screenshot shows the OneCompiler web IDE interface. The top bar includes the OneCompiler logo, an 'Upgrade' button, an 'AI' button, a 'Go' dropdown, a 'Run' button, and icons for settings, search, and save. The main editor displays a Go file named 'main.go' with the following code:

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     x := 10
7
8     if x > 5 {
9         x := x * 2
10        fmt.Println("dentro:", x)
11    }
12
13    fmt.Println("fora:", x)
14 }
```

The closing brace of the `main` function on line 14 is highlighted in yellow. To the right of the editor, the 'Console' tab shows the output of the program:

```
dentro: 20
fora: 10
```

The status bar at the bottom indicates 'Ready' and shows system icons for 'Light', 'Wiki', and 'Internet'.

Nesse caso, o problema estava no uso de `:=` dentro do `if`.

O `:=` cria uma nova variável. Então, o `x` que está dentro do `if` não é exatamente o mesmo `x` que está fora.

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     x := 10
7
8     if x > 5 {
9         x = x * 2
10        fmt.Println("dentro:", x)
11    }
12
13    fmt.Println("fora:", x)
14 }
```

dentro: 20
fora: 20

Esse exercício se relaciona com atribuição e também com o funcionamento das variáveis dentro das estruturas de controle.

O que entendi foi a diferença entre criar uma nova variável e apenas mudar o valor de uma variável que já existe. O uso de = nesse caso faz com que o valor do x original seja realmente alterado.

Exercício 6 — Java

O último exercício foi:

```
1 public class Main {
2     public static void main(String[] args) {
3         int acertos = 7;
4         int total = 10;
5
6         double taxa = acertos / total * 100;
7
8         System.out.println(taxa + "%");
9     }
10 }
```

Console: 0.0%

AI Agent: 995 ms

O problema acontece porque `acertos` e `total` são números inteiros. Então, antes de chegar no `double`, o Java faz:

$$7 / 10 = 0$$

Depois esse 0 é convertido para `double`, ficando 0.0.

```
1 public class Main {
2     public static void main(String[] args) {
3         int acertos = 7;
4         int total = 10;
5
6         double taxa = (double) acertos / total * 100;
7
8         System.out.println(taxa + "%");
9     }
10 }
```

Console: 70.0%

AI Agent: 870 ms

Esse exercício está relacionado principalmente à conversão de tipos e à mistura de tipos apresentada na aula.

Foi possível entender melhor que não basta a variável que recebe o resultado ser `double`. O tipo dos valores utilizados na operação também influencia a forma como o cálculo será realizado.

10. O que consegui aprender com os exercícios

Depois de fazer os exercícios, ficou mais fácil entender alguns conteúdos que na parte teórica podem parecer mais complicados.

Uma coisa que ficou bem clara é que erros não precisam ser grandes para mudar o resultado de um programa. Em alguns casos, foi apenas um `break`, um `;`, um operador diferente ou uma conversão de tipo que estava faltando.

Os exercícios ajudaram a entender principalmente:

- como a linguagem interpreta uma expressão;
- como os tipos podem mudar o resultado de uma operação;
- como funcionam as expressões booleanas;
- a diferença entre algumas formas de percorrer estruturas;
- o funcionamento do `switch` e do `if`;
- a diferença entre criar uma variável e alterar uma variável existente;
- como a conversão de tipos pode ser necessária em determinados cálculos.

Também percebi que testar o código ajuda bastante a entender esses conceitos. Quando o resultado aparecia diferente do esperado, era necessário olhar para o código e tentar descobrir qual detalhe estava causando aquilo.

11. Conclusão

A aula apresentou vários conceitos que fazem parte do funcionamento básico das linguagens de programação. No início, alguns deles podem parecer

apenas regras de sintaxe, mas os exercícios mostraram que essas regras realmente fazem diferença na execução do programa.

As correções ajudaram a entender melhor o conteúdo porque foi possível ver o código funcionando de uma forma, identificar o motivo do problema e depois comparar com o código corrigido.

O principal aprendizado que ficou para mim foi que é necessário prestar atenção não somente no que o código deveria fazer, mas também em como a linguagem realmente interpreta cada comando, operador, tipo e estrutura utilizada.